Chapter 2: Building a Mini-Compiler: A Simple Syntax-Directed Translator

From Source Code to Target Code

By
Dr. Adnan M. Sherif, PhD

CS432 – Fall 2026/2027

1

What We Will Learn Today

Goal: Understand the front-end phases of a compiler by building a simple translator for arithmetic expressions.

Learning Objectives:

- Explain the role of syntax-directed translation.
- Use Context-Free Grammars (CFGs) to define language syntax.
- Distinguish between Syntax-Directed Definitions and Translation Schemes.
- Implement a simple translator that converts infix expressions to postfix notation.

2

What Is Syntax-Directed Translation?

- A technique where parsing drives translation
- Uses grammar rules with semantic actions
- Produces intermediate code (e.g., postfix)

3

Using Postfix Notation

- Converts expressions like (a + b) *c into ab+c*
- Easier for code generation
- Supports stack-based evaluation

4

Grammar for Arithmetic Expressions

- Example grammar:

  E → E + T | T

  T → T * F | F

  F → (E) | id

- Left-recursive; needs to be modified

5

Eliminating Left Recursion

- Transform grammar to:

  E → T E’

  E' → + T E' | ε

  T → F T’

  T' → * F T' | ε

  F → (E) | id

6

Syntax Tree Construction

- Syntax trees represent program structure
- Internal nodes = operators
- Leaves = operands (e.g., id)

7

Postfix Code Generation

- Postfix (Reverse Polish Notation):
  a + b *c → *abc*+
- Evaluated using a stack:
  - Push a
  - Push b
  - Push c
  - Multiply
  - Add

8

A Stack-Based Evaluator

- Use a stack to evaluate postfix:
  - if token is operand → push
  - if operator → pop operands, compute, push result
- Example: ab+c*
  - Push a, b
  - Pop b, a → add → push result
  - Push c
  - Pop result and c → multiply

9

Implementation in Pseudocode

- Recursive descent parser
- Functions for each non-terminal (E, T, F)
- Build tree and/or emit postfix during parse

10

Example Walkthrough

- Input: (a + b) * c
- Tokens: (, a, +, b,), *, c
- Postfix: a b + c *
- Tree:

```
   *
  / \
 c   +
    / \
   a   b
```

11

Benefits of Syntax-Directed Translation

- Modular: closely follows grammar
- Clear mapping to code generation
- Enables intermediate code for optimization

12

Using Lex and Yacc

- Lex and Yacc are tools that generate lexical analyzers and parsers, respectively.
- They were introduced in 1975 and greatly simplify the process of writing a compiler.
- The use of these tools assumes a working knowledge of C programming and data structures like linked lists and trees.

13

The Compiler Building Blocks

1.Lexical Analyzer (Scanner):

- Reads source code.
- Converts strings of characters into a stream of tokens.
- Example: `a = b + c * d` becomes `id1 = id2 + id3 * id4`

2.Syntax Analyzer (Parser):

- Analyzes the stream of tokens from the lexical analyzer.
- Creates a hierarchical structure called a syntax tree.
- This tree represents grammatical structure, including operator precedence.

3.Code Generator:

- Performs a depth-first walk of the syntax tree to generate code.
- The output can be assembly code or machine code.

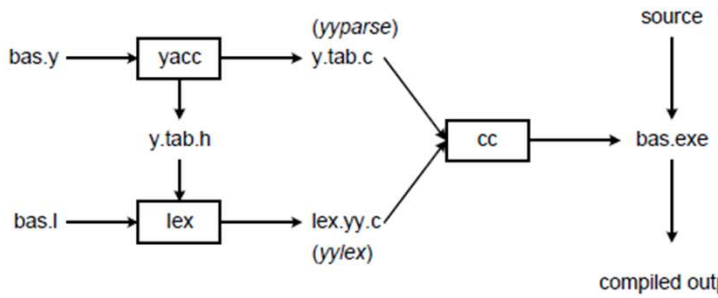
14

How Lex and Yacc Work Together

- **yacc** reads your grammar rules (bas.y) to create a parser (y.tab.c) and a header file of token definitions (y.tab.h).
- **lex** reads your pattern rules (bas.l) and the token definitions (y.tab.h) to create a lexical analyzer (lex.yy.c).
- A C compiler (cc) compiles and links the generated code to create the final executable (bas.exe).
- When the program runs, the parser (yyparse) calls the lexer (yylex) to get tokens from the source code.

15

How Lex and Yacc Work Together




```
yacc -d bas.y                    # create y.tab.h, y.tab.c
lex bas.l                        # create lex.yy.c
gcc lex.yy.c y.tab.c -obas.exe   # compile/link
```

16

Deep Dive into Lex - Theory

- **What it does:** Lex generates C code for a lexical analyzer.
- **How it works:** It uses patterns defined by regular expressions to match strings in the source input.
- **Finite State Automaton (FSA):**Any regular expression can be represented as a Finite State Automaton.
  - An FSA has a start state, transitions between states based on input characters, and one or more accepting states.
  - Lex translates the regular expressions into a program that mimics an FSA.
  - Limitation: Lex has no stack, so it cannot be used to recognize nested structures like parentheses. This is a job for a parser like Yacc.

17

Lex in Practice - Structure & Rules

Expression	*Matches*
`abc`	`abc`
`abc*`	`ab, abc, abcc, abccc, …`
`abc+`	`abc, abcc, abccc, …`
`a(bc)+`	`abc, abcbc, abcbcbc, …`
`a(bc)?`	`a, abc`
`[abc]`	`a, b, c`
`[a-z]`	any letter, `a` through `z`
`[a\-z]`	`a, -, z`
`[-az]`	`-, a, z`
`[A-Za-z0-9]+`	one or more alphanumeric characters
`[ \t\n]+`	whitespace
`[^ab]`	anything except: `a, b`
`[a^b]`	`a, ^, b`
`[a|b]`	`a, |, b`
`a|b`	`a or b`

Metacharacter	*Matches*
`.`	any character except newline
`\n`	newline
`*`	zero or more copies of preceding expression
`+`	one or more copies of preceding expression
`?`	zero or one copy of preceeding expression
`^`	beginning of line
`$`	end of line
`a|b`	a or b
`(ab)+`	one or more copies of ab (grouping)
`"a+b"`	literal “a+b” (C escapes still work)
`[]`	character class

18

Lex in Practice - Structure & Rules

- A Lex file is divided into three sections, separated by %%
- Definitions: C code copied to the top of the generated file (in %{ ... %}), and substitutions for simplifying patterns.
- Rules: A set of patterns and their associated C actions.
- Subroutines: User-written C code, including a main() and yywrap() function.



19

Lex in Practice - Structure & Rules – Example

```
%{
        int yylineno;
%}
%%
^(.*)\n  printf("%4d\t%s", ++yylineno, yytext);
%%
int  yywrap() { return 1; }
int main(int argc, char *argv[]) {
        yyin = fopen(argv[1], "r");
        yylex();
        fclose(yyin);
}
```

yytext is a pointer to the matched string.
yyleng is the length of the matched string
yywrap indicates if lines should be wrapped

20

Deep Dive into Yacc - Theory

- **What it does:** Yacc generates C code for a syntax analyzer (parser).
- **How it works:** Grammars are described using a variant of Backus Naur Form (BNF).
- **BNF Grammar Example:**
  E -> E + E
  E -> E * E
  E -> id
- E is a nonterminal (appears on the left side).
- +, *, and id are terminals (tokens from Lex).
- **Parsing Method:** Yacc uses a bottom-up, or shift-reduce, parsing method with a stack.
  - Shift: Move the next token onto the parse stack.
  - Reduce: When tokens on top of the stack match the right-hand side of a grammar rule, replace them with the rule's left-hand side nonterminal.

21

Yacc in Practice - Conflicts and Actions

- Conflicts: Sometimes a grammar is ambiguous.
- Shift-Reduce Conflict: Yacc can't decide whether to shift the next token or reduce a rule. This often relates to operator precedence. Yacc's default is to shift.
- Reduce-Reduce Conflict: Yacc can't decide which of two possible rules to reduce. Yacc's default is to use the first rule listed.
- Resolving Ambiguity:You can rewrite the grammar, or provide directives. %left, %right, and %nonassoc declare associativity and precedence for tokens.
- Actions: C code in {} associated with a rule.
  - $$ refers to the value of the resulting nonterminal.
  - $1, $2, etc., refer to the values of the terms on the right-hand side.
    - Example: expr: expr '+' expr { $$ = $1 + $3; }

22

Calculator Example - Handling Data Types

- Since a token can represent different kinds of data (an integer, a variable index, a pointer to a tree node), Yacc's value stack (yylval) must support multiple types.
- This is done using a %union declaration in Yacc's definition section:

```
%union {
  int iValue;      /* integer value */
  char sIndex;     /* symbol table index */
  nodeType *nPtr;  /* node pointer */
};
```

Tokens and nonterminals are then associated with a specific type from the union:

```
%token <iValue> INTEGER
%type <nPtr> expr
```

23

Calculator Example - Building the Syntax Tree

- Tree nodes are allocated in the actions of the grammar rules.
- Leaf nodes are created for constants and variables:

expr: INTEGER { $$ = con($1); }

- Internal (operator) nodes are created to link sub-trees:expr:

```
expr '+' expr { $$ = opr('+', 2, $1, $3); }
```

- Once a complete statement is parsed, the tree is passed to a function (ex()) for processing (interpretation or compilation).

24

Conclusion

- Lex and Yacc are powerful utilities for simplifying compiler construction.
- Lex handles pattern matching (lexical analysis).
- Yacc handles grammar and structure (syntax analysis).
- By understanding how to combine them, you can build sophisticated language processors, from calculators to full compilers.

25